

Course 4: Deep Learning



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

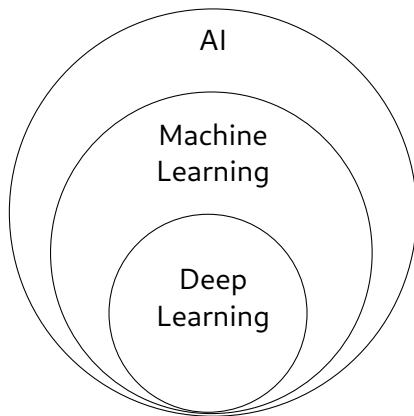
Last session

- 1 Unsupervised learning - discover structure from unlabeled data
- 2 Clustering
- 3 Decomposition
- 4 Manifold learning
- 5 Evaluation
- 6 Feature cleaning

Today's session

- Multi-Layer Perceptron
- Convolutional Neural Networks
- Transformers
- Deep Learning today

Where does Deep Learning sit?



Adapted from *Deep Learning*, 2016 by A. Courville, I. Goodfellow and Y. Bengio

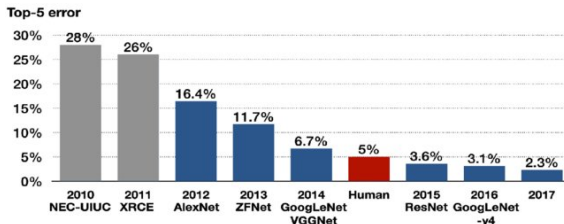
Deep Learning is a **family of models**, not a learning framework: it can be **supervised** (session 2), **unsupervised** (session 3), **self-supervised** (session 5), ...

Deep Learning in a nutshell (1/2)

Tentative definition

- Using *deep* Artificial Neural Networks
- Optimized on *large* datasets.

We generally talk about “Neural Networks” instead of “**Artificial** Neural Networks”, but the latter is the most accurate terminology!

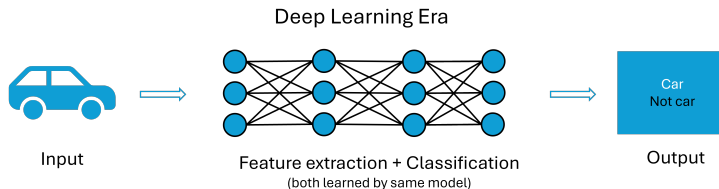
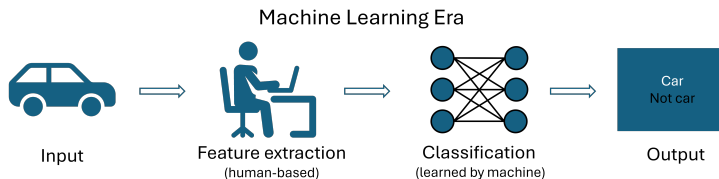


The hype really started in 2012 due to impressive results in image classification.

Source: Kang, D. Y., Duong, H. P., & Park, J. C. (2020). Application of deep learning in dentistry and implantology. *Journal of implantology and applied sciences*, 24(3), 148-181.

Deep Learning in a nutshell (2/2)

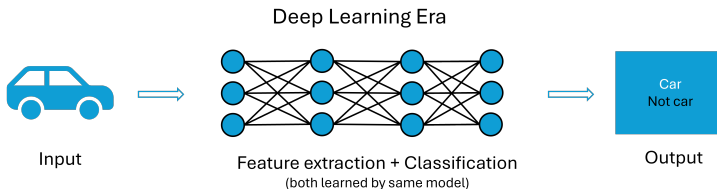
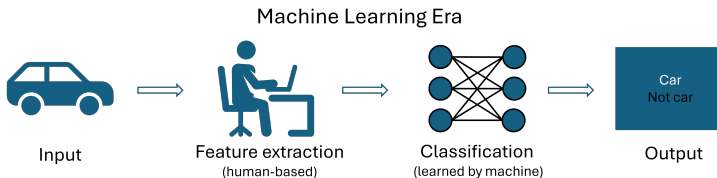
No more hand-crafted features: the **representation is learned**, jointly with the decision.



Inspired from <https://www.softwaretestinghelp.com/data-mining-vs-machine-learning-vs-ai/>

Deep Learning in a nutshell (2/2)

No more hand-crafted features: the **representation is learned**, jointly with the decision.



The price to pay: a lot (a lot) of data.

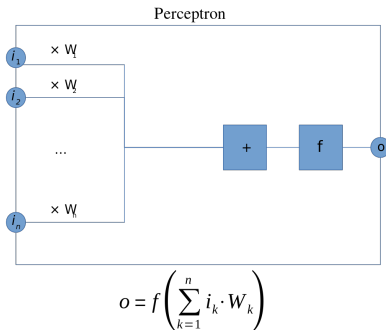
Outline

- 1 Multi-layer Perceptron
- 2 Convolutional Neural Networks (CNN)
- 3 Transformers
- 4 Deep Learning today
- 5 Lab session

Perceptron

1943, first known implementation in 1957

⇒ a weighted sum followed by a nonlinearity **f**; weights **W** are learned.



Source: By Mat the w at English Wikipedia, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=23766733>

Perceptron

1943, first known implementation in 1957

⇒ a weighted sum followed by a nonlinearity $\mathbf{f}$; weights W are learned.

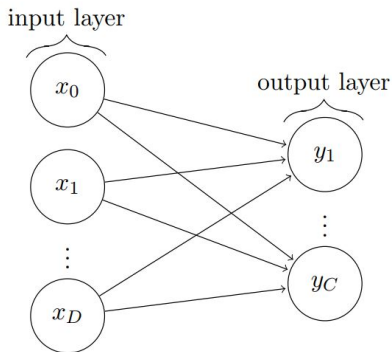


Figure: The arrows represent the weights W , and the circles represent the activation function $\mathbf{f}$.

Optimizing the weights

Loss

- Prediction: $\hat{\mathbf{y}} = f\left(\sum_{d=0}^D x_d W_d\right)$
- Ground truth: $\mathbf{y}$
- Loss (one example): $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
(ex: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$)
- Loss (i examples): $J(W) = \sum_i \mathcal{L}(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)})$

Gradient descent

- Compute the gradient: $\frac{\partial J(W)}{\partial W}$ (high dimensional derivative)
- Update weights: $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$

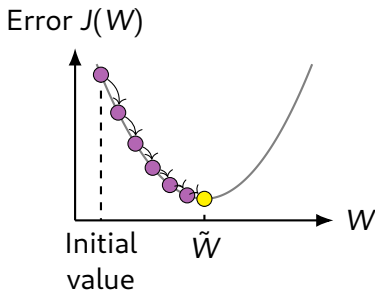
Gradient descent

Intuition behind the gradient descent

Update is given as: $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$

- $\frac{\partial J(W)}{\partial W}$ gives the direction
- η gives the size of the step

Well-chosen step: converges to $\tilde{W}$



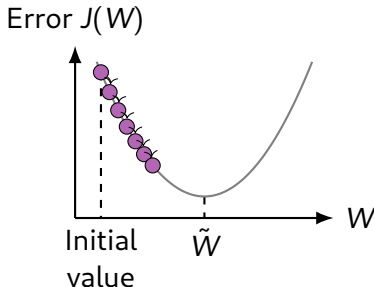
Gradient descent

Intuition behind the gradient descent

Update is given as: $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$

- $\frac{\partial J(W)}{\partial W}$ gives the direction
- η gives the size of the step

Too small a step: converges, but very slowly



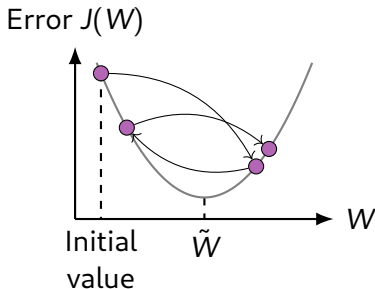
Gradient descent

Intuition behind the gradient descent

Update is given as: $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$

- $\frac{\partial J(W)}{\partial W}$ gives the direction
- η gives the size of the step

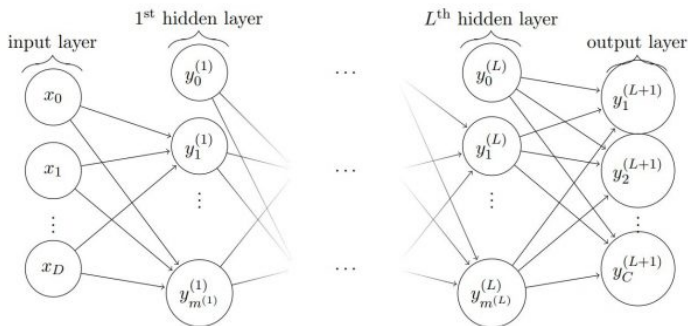
Too large a step: oscillates, may never converge



Multi-Layer Perceptron

Multi-Layer Perceptron (= *fully-connected* network)

- Stacking Perceptrons.
- The **deep** term comes from this stacking
- **Prediction:** $\hat{\mathbf{y}} = f(W^{(L)} \dots f(W^{(2)} f(W^{(1)} \mathbf{x})))$



Definitions

$$\mathbf{h}^{(l+1)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l)} + \mathbf{b}^{(l)}) = \phi^{(l)}(\mathbf{h}^{(l)})$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$

- Each building block $\mathbf{h}^{(l+1)}$ is called a **layer**.
- One element i of a layer ($h_i^{(l)}$) is called a **neuron** (both as input and output).
- The nonlinear function f is called the **activation function**.
- $\mathbf{W}^{(l)}$ are called **weights**.
- $\mathbf{b}^{(l)}$ is called the **bias**.

Note: while each layer can have a different activation function f , it is standard that each layer uses the same.

Definitions

$$\mathbf{h}^{(l+1)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l)} + \mathbf{b}^{(l)}) = \phi^{(l)}(\mathbf{h}^{(l)})$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$

- Each building block $\mathbf{h}^{(l+1)}$ is called a **layer**.
- One element i of a layer ($h_i^{(l)}$) is called a **neuron** (both as input and output).
- The nonlinear function f is called the **activation function**.
- $\mathbf{W}^{(l)}$ are called **weights**.
- $\mathbf{b}^{(l)}$ is called the **bias**.

Note: while each layer can have a different activation function f , it is standard that each layer uses the same.

Definitions

$$\mathbf{h}^{(l+1)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l)} + \mathbf{b}^{(l)}) = \phi^{(l)}(\mathbf{h}^{(l)})$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$

- Each building block $\mathbf{h}^{(l+1)}$ is called a **layer**.
- One element i of a layer ($h_i^{(l)}$) is called a **neuron** (both as input and output).
- The nonlinear function f is called the **activation function**.
- $\mathbf{W}^{(l)}$ are called **weights**.
- $\mathbf{b}^{(l)}$ is called the **bias**.

Note: while each layer can have a different activation function f , it is standard that each layer uses the same.

Definitions

$$\mathbf{h}^{(l+1)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l)} + \mathbf{b}^{(l)}) = \phi^{(l)}(\mathbf{h}^{(l)})$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$

- Each building block $\mathbf{h}^{(l+1)}$ is called a **layer**.
- One element i of a layer ($h_i^{(l)}$) is called a **neuron** (both as input and output).
- The nonlinear function f is called the **activation function**.
- $\mathbf{W}^{(l)}$ are called **weights**.
- $\mathbf{b}^{(l)}$ is called the **bias**.

Note: while each layer can have a different activation function f , it is standard that each layer uses the same.

Definitions

$$\mathbf{h}^{(l+1)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l)} + \mathbf{b}^{(l)}) = \phi^{(l)}(\mathbf{h}^{(l)})$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$

- Each building block $\mathbf{h}^{(l+1)}$ is called a **layer**.
- One element i of a layer ($h_i^{(l)}$) is called a **neuron** (both as input and output).
- The nonlinear function f is called the **activation function**.
- $\mathbf{W}^{(l)}$ are called **weights**.
- $\mathbf{b}^{(l)}$ is called the **bias**.

Note: while each layer can have a different activation function f , it is standard that each layer uses the same.

Definitions

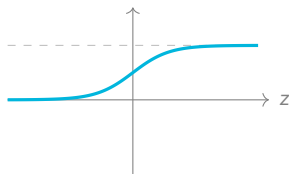
$$\mathbf{h}^{(l+1)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l)} + \mathbf{b}^{(l)}) = \phi^{(l)}(\mathbf{h}^{(l)})$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$

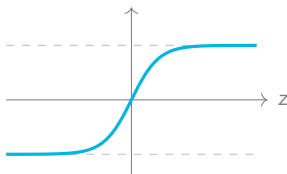
- Each building block $\mathbf{h}^{(l+1)}$ is called a **layer**.
- One element i of a layer ($h_i^{(l)}$) is called a **neuron** (both as input and output).
- The nonlinear function f is called the **activation function**.
- $\mathbf{W}^{(l)}$ are called **weights**.
- $\mathbf{b}^{(l)}$ is called the **bias**.

Note: while each layer can have a different activation function f , it is standard that each layer uses the same.

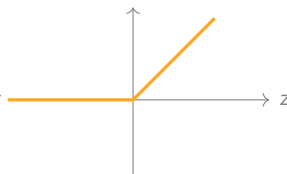
The activation function



sigmoid
 $\sigma(z) = 1/(1 + e^{-z})$



tanh
 $\tanh(z)$



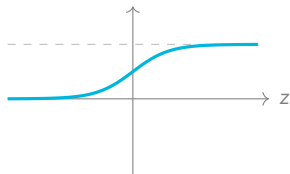
ReLU
 $\max(0, z)$
the default one (your lab!)

Why can we not simply drop f ?

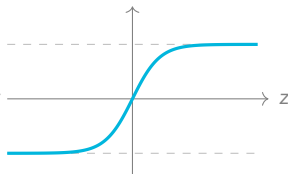
$$W^{(2)} \left(W^{(1)} \mathbf{x} \right) = \underbrace{\left(W^{(2)} W^{(1)} \right)}_{\text{one single matrix}} \mathbf{x}$$

Without f , 100 stacked layers $\approx$ 1 layer: depth would be useless.

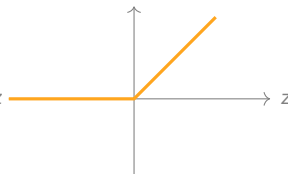
The activation function



sigmoid
 $\sigma(z) = 1/(1 + e^{-z})$



tanh
 $\tanh(z)$



ReLU
 $\max(0, z)$
the default one (your lab!)

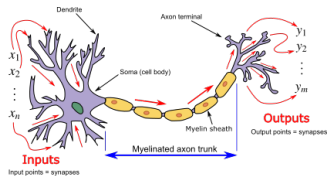
Why can we not simply drop f ?

$$W^{(2)} \left(W^{(1)} \mathbf{x} \right) = \underbrace{\left(W^{(2)} W^{(1)} \right)}_{\text{one single matrix}} \mathbf{x}$$

Without f , 100 stacked layers $\approx$ 1 layer: depth would be useless.

Why is it called Neural Network?

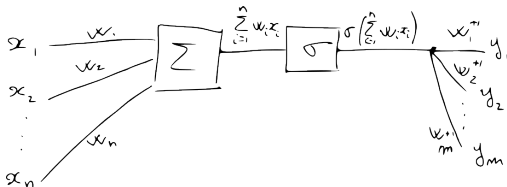
“Neurons” may be seen as **loosely** inspired from the human brain.



Attribution: Egm4313.s12 at English Wikipedia,

[https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)#/media/](https://en.wikipedia.org/wiki/Neural_network_(machine_learning)#/media/File:Neuron3.png)

File:Neuron3.png



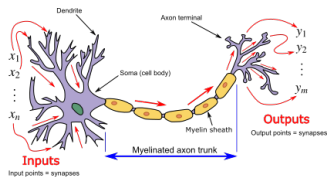
Detail of one neuron in an Artificial Neural Network.

* Late note: I forgot the bias in the graphic.

This is an **analogy**, artificial neural networks are **not** following the human brain (in general).

Why is it called Neural Network?

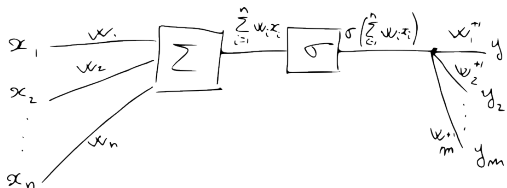
“Neurons” may be seen as **loosely** inspired from the human brain.



Attribution: Egm4313.s12 at English Wikipedia,

[https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)#/media/](https://en.wikipedia.org/wiki/Neural_network_(machine_learning)#/media/File:Neuron3.png)

File:Neuron3.png



Detail of one neuron in an Artificial Neural Network.

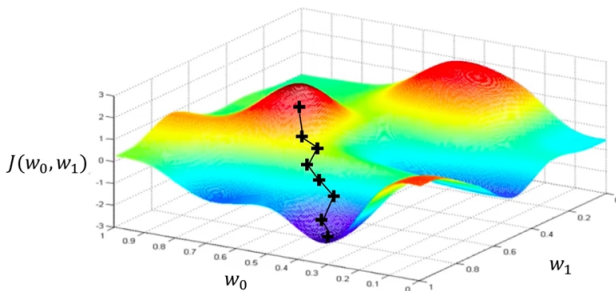
* Late note: I forgot the bias in the graphic.

This is an **analogy**, artificial neural networks are **not** following the human brain (in general).

Gradient descent for deep neural networks

Backpropagation

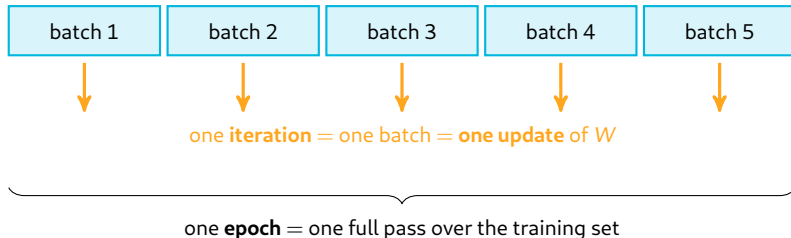
- Gradient descent for all layers (chain rule).
- Simplified equation: $\frac{\partial J(W)}{\partial W} = \frac{\partial J(W)}{\partial W^{(L)}} \frac{\partial W^{(L)}}{\partial W^{(L-1)}} \frac{\partial W^{(L-1)}}{\partial W^{(L-2)}} \dots \frac{\partial W^{(2)}}{\partial W^{(1)}}$
- The error **backpropagates** through the network (reverse path)
- Computationally efficient, but finds a local minimum (at best)



Source: <http://introtodeeplearning.com/>

Batch, iteration, epoch

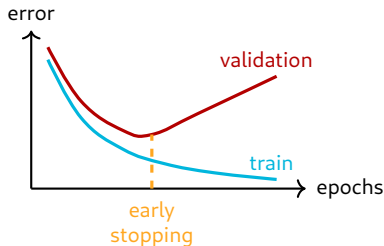
Training set, shuffled and cut into **batches**:



Why not use all the data at once?

- It would not fit in memory,
- One update per epoch would be far too slow.

Overfitting strikes back (Session 2)



Millions of parameters $\Rightarrow$ huge variance

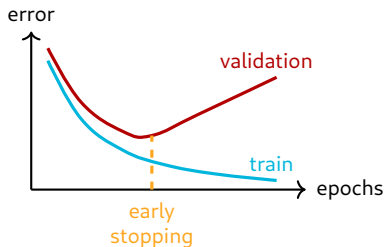
A deep network can **memorize** its training set (even random labels!).

Mitigating overfitting

- **More data** (the best cure),
- **Data augmentation**: rotate/crop/noise,
- **Dropout**: randomly switch off neurons,
- **Early stopping**: keep the best epoch,
- **Weight decay**: penalize large weights.

Always watch the validation curve!

Overfitting strikes back (Session 2)



Millions of parameters $\Rightarrow$ huge variance

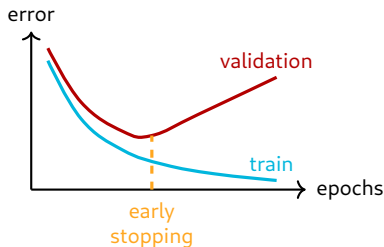
A deep network can **memorize** its training set (even random labels!).

Mitigating overfitting

- **More data** (the best cure),
- **Data augmentation**: rotate/crop/noise,
- **Dropout**: randomly switch off neurons,
- **Early stopping**: keep the best epoch,
- **Weight decay**: penalize large weights.

Always watch the validation curve!

Overfitting strikes back (Session 2)



Millions of parameters $\Rightarrow$ huge variance

A deep network can **memorize** its training set (even random labels!).

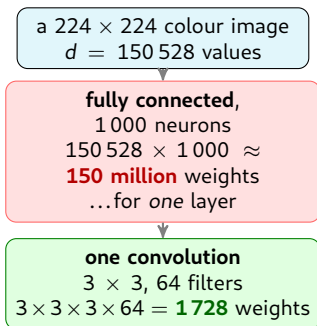
Mitigating overfitting

- **More data** (the best cure),
- **Data augmentation**: rotate/crop/noise,
- **Dropout**: randomly switch off neurons,
- **Early stopping**: keep the best epoch,
- **Weight decay**: penalize large weights.

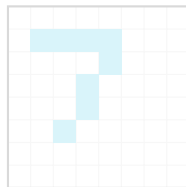
Always watch the validation curve!

Limits of the MLP

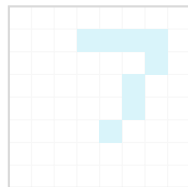
1. Too many parameters



2. No notion of structure



original



shifted by 2 pixels

Same object for you...

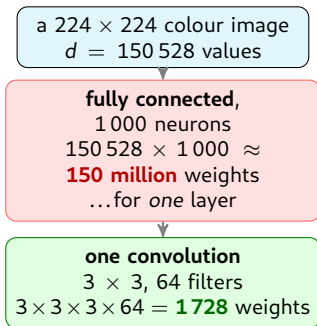
but once **vectorized**, two **very distant** vectors.

⇒ exactly the problem we met on MNIST in Session 3!

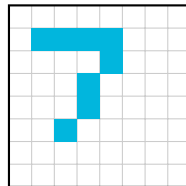
The MLP assumes nothing about the input... which is the problem.

Limits of the MLP

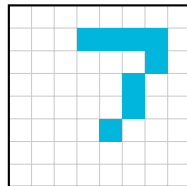
1. Too many parameters



2. No notion of structure



original



shifted by 2 pixels

Same object for you...

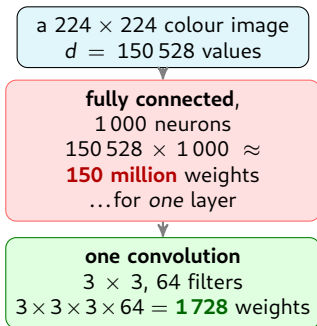
but once **vectorized**, two **very distant** vectors.

⇒ exactly the problem we met on MNIST in Session 3!

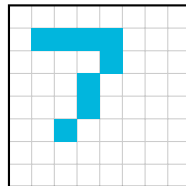
The MLP assumes nothing about the input... which is the problem.

Limits of the MLP

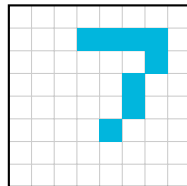
1. Too many parameters



2. No notion of structure



original



shifted by 2 pixels

Same object for you...

but once **vectorized**, two **very distant** vectors.

⇒ exactly the problem we met on MNIST in Session 3!

The MLP assumes nothing about the input... which is the problem.

Outline

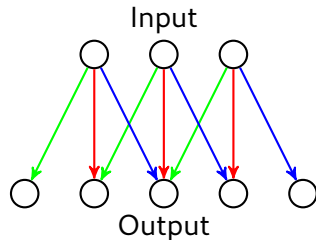
- 1 Multi-layer Perceptron
- 2 Convolutional Neural Networks (CNN)
- 3 Transformers
- 4 Deep Learning today
- 5 Lab session

Convolutional Neural Network (1/5): the principle

Two ideas, motivated for image processing

- **Local:** a kernel looks at a small neighbourhood at a time
⇒ few weights,
- **Shared:** the **same** weights are applied everywhere
⇒ **translation equivariance**.

Convolutional layer



$$\begin{pmatrix} w_1 & w_2 & w_3 & 0 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 & 0 \end{pmatrix}$$

Convolutional Neural Network (2/5): the operation

Example of 2D convolution:

$$\begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} * \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 4 & 3 & 4 & 1 \\ 1 & 2 & 4 & 3 & 3 \\ 1 & 2 & 3 & 4 & 1 \\ 1 & 3 & 3 & 1 & 1 \\ 3 & 3 & 1 & 1 & 0 \end{pmatrix}$$

$I \qquad K \qquad I * K$

Source: <https://tex.stackexchange.com/questions/437007/drawing-a-convolution-with-tikz>

Convolutional Neural Network (2/5): the operation

Example of 2D convolution:

$$\begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} * \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 4 & 3 & 4 & 1 \\ 1 & 2 & 4 & 3 & 3 \\ 1 & 2 & 3 & 4 & 1 \\ 1 & 3 & 3 & 1 & 1 \\ 3 & 3 & 1 & 1 & 0 \end{pmatrix}$$

$I \qquad K \qquad I * K$

Source: <https://tex.stackexchange.com/questions/437007/drawing-a-convolution-with-tikz>

Convolutional Neural Network (2/5): the operation

Example of 2D convolution:

$$\begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} * \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 4 & 3 & 4 & 1 \\ 1 & 2 & 4 & 3 & 3 \\ 1 & 2 & 3 & 4 & 1 \\ 1 & 3 & 3 & 1 & 1 \\ 3 & 3 & 1 & 1 & 0 \end{pmatrix}$$

$I \qquad K \qquad I * K$

Source: <https://tex.stackexchange.com/questions/437007/drawing-a-convolution-with-tikz>

Convolutional Neural Network (3/5): pooling

Example of 2D pooling:

1	2	3	1
1	1	1	1
2	3	1	6
8	1	4	5



maxpool, kernel 2, stride 2

Convolutional Neural Network (3/5): pooling

Example of 2D pooling:

1	2	3	1
1	1	1	1
2	3	1	6
8	1	4	5



maxpool, kernel 2, stride 2

Convolutional Neural Network (3/5): pooling

Example of 2D pooling:

1	2	3	1
1	1	1	1
2	3	1	6
8	1	4	5



2	3
8	

maxpool, kernel 2, stride 2

Convolutional Neural Network (3/5): pooling

Example of 2D pooling:

1	2	3	1
1	1	1	1
2	3	1	6
8	1	4	5



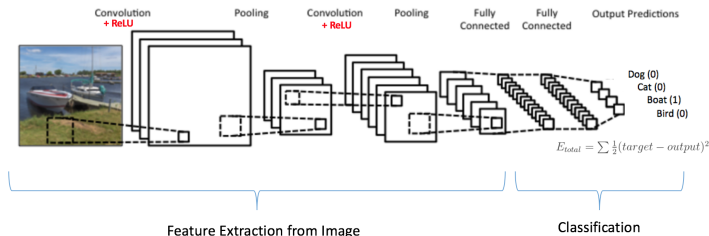
2	3
8	6

maxpool, kernel 2, stride 2

Convolutional Neural Network (4/5): and repeat...

A CNN is mainly *Convolution + Pooling*, repeated

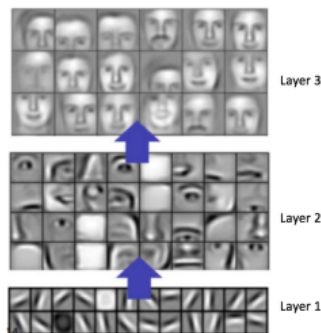
- Progressively: **smaller** maps, **more** channels, Ending with a small MLP to decide.
- ...But many other components may be added! (batch norm, dropout, skip connections, concatenation, ...)
- Not only for images: **1D** convolutions for audio, **3D** for video, ...



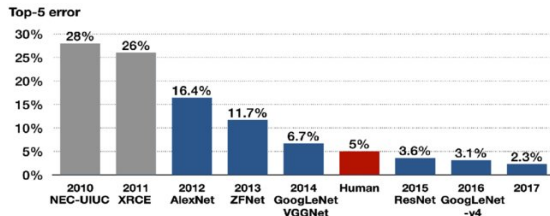
Convolutional Neural Network (5/5): what is learned?

Why convolutions?

- Kernels capture important information in images,
- They become more and more **complex** (and more **abstract**) with the depth.



So, what really happened in 2012?



Not a new idea: a combination of four ingredients

- Convolutional networks (1989!)
- A very large dataset (ImageNet)
- **GPUs** for computation
- Clever tricks (data augmentation, ReLU, dropout)

Source: Kang, D. Y., Duong, H. P., & Park, J. C. (2020). Application of deep learning in dentistry and implantology. *Journal of implantology and applied sciences*, 24(3), 148-181.

Human eval: Russakovsky et al., ImageNet Large Scale Visual Recognition Challenge, <https://arxiv.org/pdf/1409.0575.pdf>

Outline

- 1 Multi-layer Perceptron
- 2 Convolutional Neural Networks (CNN)
- 3 Transformers
- 4 Deep Learning today
- 5 Lab session

Standard architecture nowadays

- No convolution
- Based on *attention*: what should be important for context?
- Used for text, image, audio, ...

Standard architecture nowadays

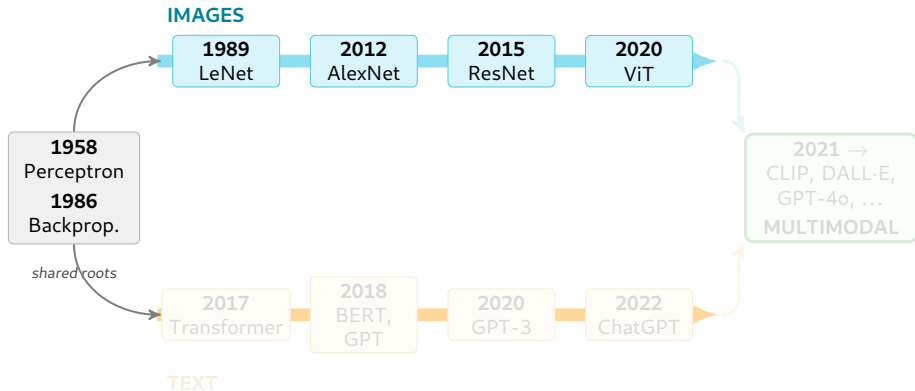
- No convolution
- Based on *attention*: what should be important for context?
- Used for text, image, audio, ...

Will be presented in the next class (foundation models)!

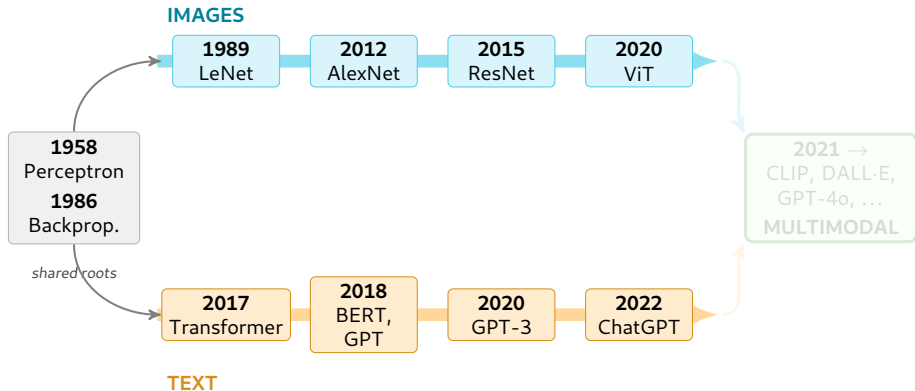
Outline

- 1 Multi-layer Perceptron
- 2 Convolutional Neural Networks (CNN)
- 3 Transformers
- 4 Deep Learning today**
- 5 Lab session

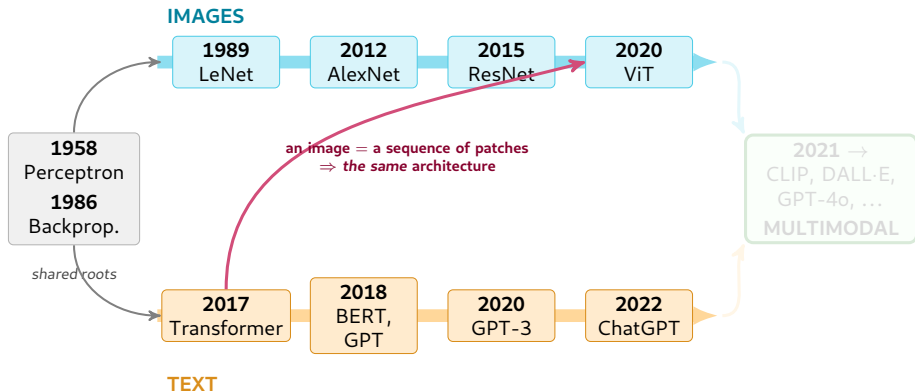
Images and Text drives the AI revolution



Images and Text drives the AI revolution

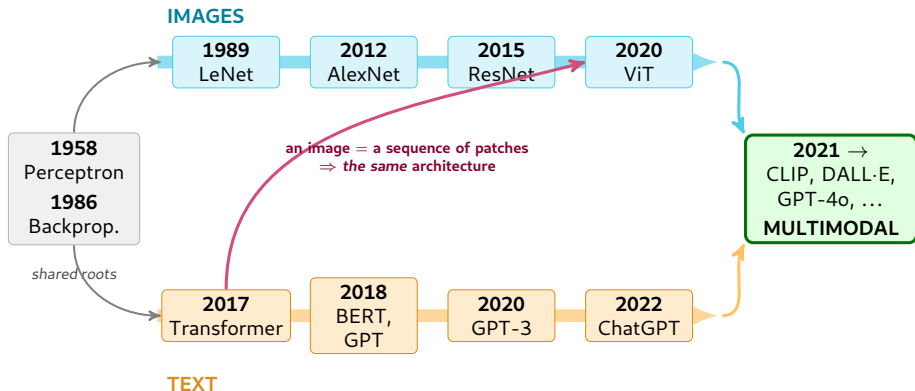


Images and Text drives the AI revolution



An additional note: Recent models all use the Transformer architecture!

Images and Text drives the AI revolution



An additional note: Recent models all use the Transformer architecture!

Models keep growing in size

Number of parameters

- AlexNet (2012): 62 Million
- GPT-3 (2020): 175 Billion

⇒ a factor $\approx 3\,000$ in 8 years.

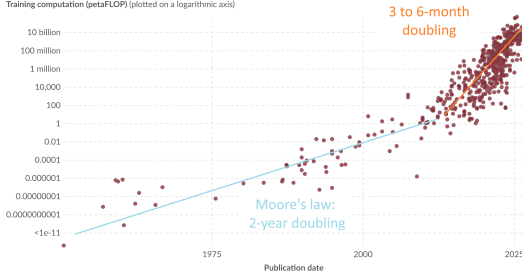
The training cost follows: data,
GPU-hours, energy.

You won't have to train
such a model! You can
reuse one.

Computation used to train notable artificial intelligence systems

Computation is measured in total petaFLOP, which is 10^{15} floating-point operations¹. Estimated from AI literature, albeit with some uncertainty. Estimates are expected to be accurate within a factor of 2, or a factor of 5 for recent undisclosed models like GPT-4.

Training computation (petaFLOP) (plotted on a logarithmic axis)



Data source: Epoch AI (2026)

OurWorldInData.org/artificial-intelligence | CC BY

¹ Floating-point operation A floating-point operation (FLOP) is a type of computer operation. One FLOP represents a single arithmetic operation involving floating-point numbers, such as addition, subtraction, multiplication, or division.

Computing power used to train AI models, over time (session 1).

Note the **logarithmic** vertical scale.

Models keep growing in size

Number of parameters

- AlexNet (2012): 62 Million
- GPT-3 (2020): 175 Billion

⇒ a factor $\approx 3\,000$ in 8 years.

The training cost follows: data, GPU-hours, energy.

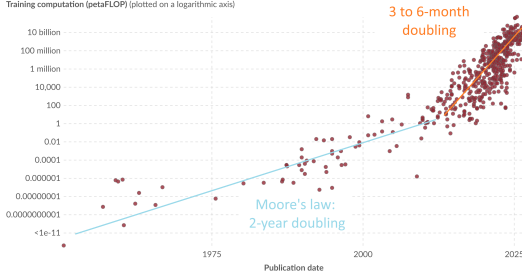
You won't have to train such a model! You can reuse one.

Computation used to train notable artificial intelligence systems

Our World In Data

Computation is measured in total petaFLOP, which is 10^{15} floating-point operations¹. Estimated from AI literature, albeit with some uncertainty. Estimates are expected to be accurate within a factor of 2, or a factor of 5 for recent undisclosed models like GPT-4.

Training computation (petaFLOP) (plotted on a logarithmic axis)



Data source: Epoch AI (2026)

OurWorldInData.org/artificial-intelligence | CC BY

¹ Floating-point operation A floating-point operation (FLOP) is a type of computer operation. One FLOP represents a single arithmetic operation involving floating-point numbers, such as addition, subtraction, multiplication, or division.

Computing power used to train AI models, over time (session 1).

Note the **logarithmic** vertical scale.

State of the art: what is solved, what is not

Essentially solved

- Classification on a **large, clean, public** dataset,
- Generating fluent text, plausible images, speech,
- Building a good **generic representation** of images, text, audio.

Still hard

- **Little data**, or a **specialized** domain (medical imaging, industry),
- **Transferring** to a dataset that differs from the training one,
- **Reliability**: hallucinations, robustness, calibration,
- **Cost**: data, energy, hardware.

State of the art: what is solved, what is not

Essentially solved

- Classification on a **large, clean, public** dataset,
- Generating fluent text, plausible images, speech,
- Building a good **generic representation** of images, text, audio.

Still hard

- **Little data**, or a **specialized** domain (medical imaging, industry),
- **Transferring** to a dataset that differs from the training one,
- **Reliability**: hallucinations, robustness, calibration,
- **Cost**: data, energy, hardware.

What to remember: an architecture is a *prior*

	MLP	CNN	Transformer
<i>sees the data as...</i>	 a flat vector	 a regular grid	 a set of tokens
<i>key idea</i>	everything is connected to everything	local window, weights shared everywhere	attention : the neighbourhood is learned
<i>consequence</i>	huge number of parameters, no invariance	few parameters, translation invariance	very general, but very data-hungry
<i>typical data</i>	tabular data, embeddings	images, audio, video	text, and... everything else today

Choosing an architecture = choosing what you already know (session 2!).

Conclusion

Conclusion

- **Powerful without feature extraction:** the representation is **learned**,
- They require **a lot of data** to be trained,
- **The architecture plays an important role:** it encodes what you know about your data.

Common criticisms

- Hard to **interpret**,
- **Reproduce the biases** present in the data,
- May require massive amounts of **energy**.

Going further

- Details and maths behind AI (3Blue1Brown): <https://youtu.be/aircAruvnKk>
- Intelligence and AI (M.Phi, french): <https://youtu.be/AwUpxj0DogQ>
- AI & music (A. Neely): <https://youtu.be/U8dcFhF0Dlk>

Conclusion

Conclusion

- **Powerful without feature extraction:** the representation is **learned**,
- They require **a lot of data** to be trained,
- **The architecture plays an important role:** it encodes what you know about your data.

Common criticisms

- Hard to **interpret**,
- **Reproduce the biases** present in the data,
- May require massive amounts of **energy**.

Going further

- Details and maths behind AI (3Blue1Brown): <https://youtu.be/aircAruvnKk>
- Intelligence and AI (M.Phi, french): <https://youtu.be/AwUpxj0DogQ>
- AI & music (A. Neely): <https://youtu.be/U8dcFhF0Dlk>

Conclusion

Conclusion

- **Powerful without feature extraction:** the representation is **learned**,
- They require **a lot of data** to be trained,
- **The architecture plays an important role:** it encodes what you know about your data.

Common criticisms

- Hard to **interpret**,
- **Reproduce the biases** present in the data,
- May require massive amounts of **energy**.

Going further

- Details and maths behind AI (3Blue1Brown): <https://youtu.be/aircAruvnKk>
- Intelligence and AI (M.Phi, french): <https://youtu.be/AwUpxj0DogQ>
- AI & music (A. Neely): <https://youtu.be/U8dcFhF0Dlk>

Outline

- 1 Multi-layer Perceptron
- 2 Convolutional Neural Networks (CNN)
- 3 Transformers
- 4 Deep Learning today
- 5 Lab session

Lab Session 4: Deep Learning with PyTorch

Part 1 — The training loop, with PyTorch

- **Tensors:** like `numpy` arrays, but with gradients and GPU support,
- Define a model (`nn.Module`, `nn.Linear`, `nn.Conv2d`, `ReLU`, pooling),
- The **training loop**, and it is always the same five lines: `zero_grad` → `forward` → `loss` → `backward` → `step`,
- **Train a small CNN** on CIFAR-10, and evaluate it (accuracy, per class),
- **Look inside:** display the feature maps of the first convolutional layer.

Part 2 — Your own project data

Take the same training loop, and train a first model on your project.
⇒ what is one sample, and what shape does it have? which loss? which final layer? Keep a **validation set**, plot train and validation loss, and compare with the `sklearn` baselines of sessions 2 and 3.

Link to the lab: <https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab4/>

Lab Session 4: Deep Learning with PyTorch

Part 1 — The training loop, with PyTorch

- **Tensors:** like `numpy` arrays, but with gradients and GPU support,
- Define a model (`nn.Module`, `nn.Linear`, `nn.Conv2d`, `ReLU`, pooling),
- The **training loop**, and it is always the same five lines: `zero_grad` → `forward` → `loss` → `backward` → `step`,
- **Train a small CNN** on CIFAR-10, and evaluate it (accuracy, per class),
- **Look inside:** display the feature maps of the first convolutional layer.

Part 2 — Your own project data

Take the same training loop, and train a first model on your project.
⇒ what is one sample, and what shape does it have? which loss? which final layer? Keep a **validation set**, plot train and validation loss, and compare with the `sklearn` baselines of sessions 2 and 3.

Link to the lab: <https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab4/>